

Smart RFID Door Lock System Using ESP32 and FreeRTOS

Authors:

ShengYao Liu, sliu1229/2473155
Aditi Manjunath, aditim4/2234529

Date: 03/17/2026

CSE474, Winter 2026

Introduction	3
System Design and Implementation	3
Code Structure	4
Discussion	7
Division of Work	7
Suggestions	7
Conclusions	8
References	8
Appendices	9
Total Number of hours Spent	9

Bonus: Project Demonstration Video: Youtube Link
<https://youtu.be/zeiLwfQ1mXc>

1. Introduction

Security and controlled access systems are used widely, in offices, labs, and apartments. Lock and key mechanisms are slowly being replaced with smart systems which use automated authentication control.

This project implements a Smart RFID Door Lock System using an ESP32 microcontroller running FreeRTOS. The system uses RFID cards to determine access by using user identity and current time. A DS3231 RTC is used to provide accurate timekeeping and an IR remote allows the system to switch between different access modes.

LED, I2C LCD display and buzzer are used to provide visual and audio feedback. Tasks are distributed amongst both the cores of the ESP to make sure performance is efficient.

2. System Design and Implementation

Door lock system working steps

1. The RFID card is scanned by the user using the RC522 reader
2. The system reads the UID of the card and checks if it matches an authorized card
3. DS3231 RTC gives the current time
4. The operating mode is chosen using the IR remote:
 - a. Office mode
 - b. Locked mode
 - c. Free mode
5. Access is either granted or denied based on the modes
6. The result is indicated using:
 - a. Green LED & high tone -> Access granted
 - b. Red LED & low tone -> Access denied
 - c. LCD display shows system status

FreeRTOS ensures that tasks run parallelly handling hardware inputs, system control, and user feedback effectively.

Hardware Components

Hardware modules integrated:

1. ESP32: Is the central microcontroller
 - a. Dual-core processing
 - b. Built-in FreeRTOS support
 - c. SPI and I2C communication interfaces

2. RC522 RFID Reader: Reads RFID cards using SPI interface. When a card is read its UID (unique identifier) is caught and used for authentication purposes.
3. DS3231 Real-Time Clock: Provides accurate time data using I2C bus and enables the system to enforce time-based access rules during office hours
4. IR Receiver: Receives commands from an IR remote control and specific buttons are used to switch the system between operating modes.
5. LCD Display: Provides real-time status information like:
 - a. Current system mode
 - b. Access results
 - c. RFID readiness
6. LEDs and Buzzer: Two LEDs and a buzzer provide visual and audio feedback
 - a. Green LED: Access provided
 - b. Red LED: Access denied
 - c. Buzzer tones: low- denied, high- provided

3. Code Structure

The code is classified into different modules and tasks that are implemented together to ensure the functionality of the system. The hardware components are initialized, FreeRTOS tasks are created, and inter-task communication is managed using queues and mutexes in the main function.

The code has helper functions like `uidMatch()` which verifies RFID card identity, `officeHours()` checking time restrictions, and functions like `grantAccess()` and `denyAccess()` which controls LEDs and the buzzer.

FreeRTOS tasks are created in the setup function using `xTaskCreatePinnedToCore()`, which assigns tasks to the ESP32 cores. Messages between tasks are sent via Queues, mutexes ensure that shared variables such as system mode and current time are accessed. This structure helps make the system easier to understand, debug, and maintain.

Software Architecture

The system software is implemented using FreeRTOS and multiple tasks are enabled to run parallelly on the ESP32

FreeRTOS mechanisms used:

1. Tasks: parallel execution
2. Queues: communication between the tasks
3. Mutexes: protects shared data
4. Tick based scheduling: periodic execution

Each specialized task handles specific responsibilities

FreeRTOS Task Design

The system runs 6 main tasks that are distributed across the ESP32's cores

1. RFID Task (on core 0): New RFID tasks are checked at a frequency of approx 50 Hz
 - Checks for new cards
 - Reads UID values
 - Checks UID with authorized UID
 - Sends RFID events to the Control Task
 - Adds a cooldown period to make sure repeated scans don't happen
2. IR Task(Core 0)
 - Decodes IR signals
 - Identifies command buttons
 - Switches system operating mode
 - Updates shared mode variable using a mutex
3. RTC Task (Core 1): Reads the current time every second
 - Obtains time from DS3231
 - Updates shared time variables
 - Protects time using mutex
4. Control Task (Core 1): Decision making component of the system
 - Waits for RFID scans
 - Obtains current system mode
 - Obtains current time
 - Access control logic application
 - Send results to UI and LCD tasks
5. UI Task (Core 1): Controls LEDs and Buzzer

- Receives grant/deny actions from the Control Task
 - Activates LEDs and buzzer accordingly
 - Provides feedback to the user
6. LCD Task (Core 1): LCD task handles the display output
- Displays the current system mode
 - Shows access results (GRANT or DENY)
 - Reverts to "RFID ready" after the scan cools down

Access Control Logic: Implemented in the control task

Modes:

- Mode 1: Access is granted when the RFID card matches the authorized UID +
The current time is within office hours (9 AM- 5PM)
- Mode 2: Locked Mode: Access is always denied irrespective of card or time
- Mode 3: Access is always granted

User Interface and Feedback

Visual + audio signals for status signals

LCD Display:

- Current operating mode
- Scan results
- Readiness of RFID

LED and Buzzer Signals

Grant-> Green LED-> Buzzer plays high tone

Deny-> Red LED-> Buzzer plays low tone

System Testing: Tested under different conditions

Test Case 1: Authorized Card During Office Hours

- Access Granted
- Green LED high
- LCD displays 'GRANT'

Test Case 2: Unauthorized Card

- Access Denied

- Red LED high
- LCD displays 'DENY'

Test Case 3: Locked Mode:

- Access always denied

Test Case 4: Free Mode:

- All cards have access

After verifying tests, the system applies identity and time based restrictions

4. Discussion

While implementing there were a lot of technical challenges encountered. A major challenge we faced was ensuring that synchronization between tasks running on different cores of the ESP32 was maintained. Common variables such as the current mode and current time could be used by several tasks at the same time. We used mutex locks to protect shared data.

Another problem was RFID scans being repeated when a card stayed near the reader. This was solved by adding a cooldown time that disabled scanning temporarily after a read was successful.

Debugging the hardware communication was difficult because the system used different communication protocols like SPI for the RFID reader and I2C for the RTC and LCD display. We used serial debugging to ensure that each component was initialized correctly and communicated with the ESP32 as required.

5. Division of Work

Aditi worked on implementing the hardware connections, base of the code and wrote the report. ShengYao Liu continued working on the hardware, code and finished up the code for the project. She also tested out the system and ensured it worked.

6. Suggestions:

Future improvements could allow multiple authorized RFID cards rather than a single UID. This would ensure the system is more practical and works well in the real world like offices or laboratories.

Another improvement would be the addition of WiFi which can ensure access attempts or allow remotely monitoring the system. In addition, integrating a physical door locking system like a door motor would ensure the system could control an actual door.

7. Conclusion

This project implements a **smart RFID-based access control system** using the ESP32 and FreeRTOS.

By adding RFID authentication, rules based on the time, and several modes of operation, the system has access control that is both flexible and secure. FreeRTOS ensures efficient multitasking and makes the ESP32 handle different hardware components at the same time.

The system can be further enhanced by adding:

- Several authorized RFID cards
- Logging access events
- Mobile control on apps
- A door motor

To summarize, the project showcases the efficiency of real-time embedded systems in security systems.

8. References

Arduino Documentation. <https://www.arduino.cc>

FreeRTOS Documentation. <https://www.freertos.org>

MFRC522 Library Documentation

RTCLib Library Documentation

IRremote Library Documentation

9. Appendices

Total Number of Hours Spent

Total hours spent by both students combined: 24 hours